MQTT Workshop

Protocols, Arquitecture and Practical Implementation with PubSubClient



Table of Contents

i. Fundamentals of Networking and Communications
ii. The MQTT Protocol
iii. Message Formatting
iv. Code Implementation
v. Practical Exercices

NEEEC-FEUP



Fundamentals of Networking and Communications

NEEEC-FEUP

The OSI Model

(Open Systems Interconnection)

- **What is it?**
  - A standardized conceptual model that organizes network communication functions into seven distinct modular layers.
- **Purpose:**
  - Provides a universal standard for product and software development, allowing systems from different vendors to communicate.
- **Structure:**
  - The layers work together, each responsible for a specific set of services while passing data along to the next layer.

NEEEC-FEUP

The OSI Model

APPLICATION LAYER

User Interface

Includes protocols such as HTTP, FTP, MQTT

PRESENTATION LAYER

Data Formatting

Data translation, encryption and compression

SESSION LAYER

Connection management

TRANSPORT LAYER

End-to-end data delivery

TCP - with checks

UDP - fast, but without checks

NETWORK LAYER

Routing - finding the fastest and more efficient way of sending packets from A to B via multiple networks

IP addreesses

DATALINK LAYER

Data transfer on the same local network

MAC address

PHYSICAL LAYER

Physical medium - 0s and 1s

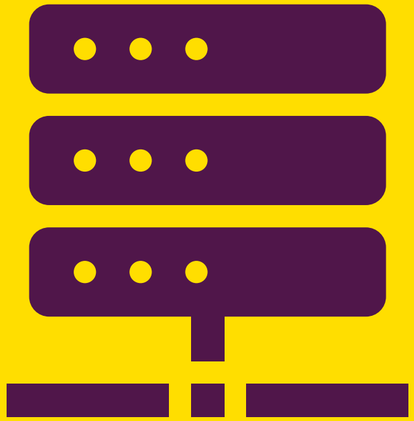
Cabes, WiFi, radio, etc...

NEEEC-FEUP

Communication Models

Characteristic	Master/Slave	Client/Server	Pub/Sub
Focus	Control of **Hardware** and Physical Devices.	**Service** and **Resource Sharing** (Data, Web).	**Message Delivery** and Decoupling.
Initiation	The **Master always** initiates communication. The **Slave** just responds.	The **Client** initiates the transaction by sending the **Request**.	**Any** device can start the communication: The **Publisher** sends; The **Subscribers** receive.
Synchronization	Usually **synchronous** (polling-based).	**Synchronous** (the **Client** waits for the **Server**'s response).	**Asynchronous** (the **Publisher** sends data and doesn't wait for the **Subscriber**).
Relationship	**Hierarchical** and **Static**. The **Slave** depends on the **Master**.	**Hierarchical**. The **Client** depends on the **Server**.	**Decoupled** and **Flexible**. Entities only depend on the **Broker**.
Knowledge of the Destination	The **Master** must know the **Slave** (address).	The **Client** must know the **Server** (IP/URL).	The **Publisher** doesn't know the **Subscriber**.
Common Protocols	Modbus, I²C, CAN Bus.	HTTP, FTP, REST.	**MQTT**, AMQP, Kafka.
IoT Suitability	Good for the **field** layer (sensors/actuators).	**Poor** (high latency and power consumption).	**Excelent** (fault tolerance and low power consumption).

NEEEC-FEUP



The MQTT Protocol

NEEEC-FEUP

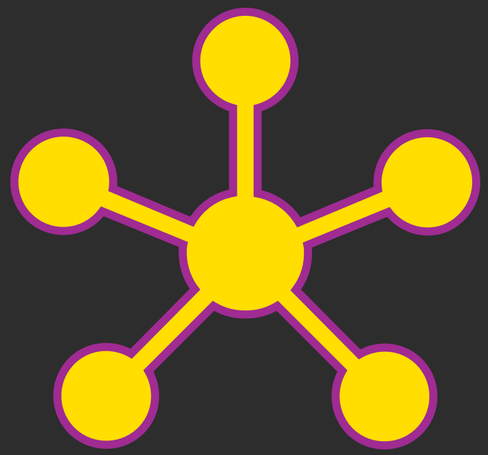
MQTT: The Lightweight Protocol

- **M**essage **Q**ueuing **T**elemetry **T**ransport
- Uses the **Pub/Sub** model, guarenteeing decoupling.
- It’s extremely **lightweight** and **efficient**. The minimum header is a message of **just 2 bytes**.
- Ideal for devices with **low bandwidth**, **high latency** and **low power** (batteries).
- Operates over **TCP/IP**.

NEEEC-FEUP

The Broker: The Center of Everything

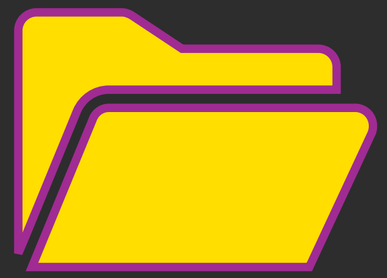
- It’s the **central intermediary** that receives every message from the **Publishers** and distributes them through the correct **Subscribers**.
- The Publisher and the Subscriber **don’t communicate directly**, only via the Broker.
- Allows the efficient connection of thousands of devices at the same time.
- It’s able to **retain** one message per topic for new devices to receive as soon as they connect.



NEEEC-FEUP

Topics: The Addressing System

- Topics define **where** messages are published and **where** they're subscribed from.
- They're organized in a **hierarchical** structure (like a file system) and they're **case sensitive**.
  - *Ex: home/kitchen/temperature*
- **Wildcards** allow for flexible subscriptions:
  - *home/**+**/temperature → the temperature of every room*
  - *home/kitchen/**#** → every sensor in the kitchen*



NEEEC-FEUP

State and Fidelity

- **Retained Messages**
  - Any message can send a *flag* that tells the Broker to **retain it** for the specified topic.
  - For example, they can guarantee that any new subscriber receives the current state immediately after subscribing to the topic.
- **Last Will and Testament**
  - A message that **a publisher or a subscriber establishes** when it connects to the broker.
  - If a **client disconnects unexpectedly**, the Broker automatically publishes the LWT message on a pre-defined topic.

NEEEC-FEUP

State and Fidelity

- **QoS - Quality of Service**
  - **QoS** defines how important is a message and the certainty that the Broker and Client give on the delivery precision.

Level	Guarantee	Short Description	Ideal for...
0	**At Most Once**	**No Guarantee of Delivery.** Fast, but the message may be **lost**. No confirmation.	**Speed** (non-critical data, such as logs).
1	**At Least Once**	**Guaranteed Delivery.** The message may be received more than once. Requires 1 acknowledgement.	**Fidelity** (critical data that tolerates duplication).
2	**Exactly Once**	**One and Only One Delivery.** Requires 4 steps.	**Maximum fidelity** (sensitive controls).

NEEEC-FEUP

State and Fidelity

- **QoS – In this Workshop**
  - On this workshop we will use a library called **“PubSubClient”**
  - This library was made to be efficient for this context, where we only use microcontrolers, we will only use **QoS 0** and **QoS 1**
- **QoS – Real life context**
  - **In industrial contexts** (PLC), normaly use **QoS 1** or **QoS 2.**
  - More powerfull platforms (like on raspberry pi computers) we can use **QoS 2**, with other libraries:
    - **Python**: [paho-mqtt, gmqtt (async e MQTT5)]
    - **C**: [libmosquitto, Eclipse Paho C (MQTT5)]
    - **C++**: [Eclipse Paho C++ (OOP)]

NEEEC-FEUP

In Summary...

Entity / Action	Topic	Function
Light Sensor (*publishes*)	devices/rasp/lights/data	Sends the current state (ON/OFF).
Command (*publishes*)	devices/rasp/lights/cmd	Sends rasp a command (to turn ON/OFF).
App (*subscribes*)	devices/+/lights/#	Listens to every command and piece of data from every light sensor.
LWT (*unexpected failure*)	devices/rasp/lights/LWT	The **Broker** publishes "Offline" if the sensor fails.

NEEEC-FEUP



Message Formatting

NEEEC-FEUP

Message Formatting

- MQTT send only *bytes* (the message *payload)*. If we want to send **different pieces of data** in a single message (temperature, humidity, etc.), we need a **serialization format**.
- **Common Serialization Formats**:
  - **Binary**: It's the **most efficient** (*smallest*) because it send only a pure *byte* array (*Ex: registers*), but both the Sender and the Receiver must know the exact *byte* structure (*order and size*) ahead of time.
  - **TXT/CSV**: Simple and readable, but lacking clear structure.
  - **JSON**: The most common due to its **simplicity** and **readability** - the *de facto* standard on the web.

NEEEC-FEUP

JSON: The Format for Readability

- **Human Readability**: JSON uses **key-value** (“key”: value) pairs and it’s text-based.
- This means we can easily inspect what a device is sending, which is crucial during the **development** and ***debugging*** stage.



NEEEC-FEUP

JSON: The Format for Readability

```json
{
  "temperature": 23,
  "battery": {"voltage": 3.71, "percent": 87}
}
```

```json
{
  "last_temperatures": [23, 23, 22, 21]
}
```

NEEEC-FEUP

JSON Is Good, But...

- It has a lot of overhead, since it's only a string formatted according to a specific standard.
- Serialization is **slow** for machines.
  - We humans can quickly understand the type of each value (*int*, *float*, *string*, *etc*...).
  - A machine cannot...
- So how can we achieve **performance**?
  - Minimize the message's size.
  - Make it native for the machine (*binary*) *so parsing* is instantaneous and doesn't require conversion.

NEEEC-FEUP

MessagePack: The Binary JSON

- **MessagePack**, or **msgPack**, is a binary serialization format, which means it transforms its data in a compact sequence of **pure *bytes***.
- The library we will be using does the heavy lifting for us:
  - It **automatically converts** data from **JSON/Objeto** (*which we code with*) to **MessagePack** (*which the machine sends and receives*) and vice-versa.
  - **The result**: We get the **performance of binary**, while keeping the **simplicity** of working with JSON-like structures in the code.

NEEEC-FEUP

- Time for a quick demonstration

NEEEC-FEUP



Code Implementation

NEEEC-FEUP

Comparison of Microcontrollers with Wireless Capabilities

Board	Cost/Benefit	Processing Capacity	Ideal for...
ESP8266	Very low	Single core (80MHz)	Simple telemetry projects (*only sending data*).
ESP32	Great	**Dual Core** (up to 240MHz) and more memory.	Robust applications, *gateways*, heavier Bluetooth and processing.
Pico W (RP2040)	Great	**Dual Core** (133MHz) and excellent I/O.	Ideal balance for MQTT, RPC and management of multiplie concurrent processes.

NEEEC-FEUP

The Raspberry Pi Pico W

- In this Workshop we will be using the *Raspberry Pi Pico W*
- The *Pico W* supports the **Arduino framework**, but **not natively**.
- The following setup is necessary:
  1. Copy the folloring URL: https://github.com/earlephilhower/arduino-pico/releases/download/global/package_rp2040_index.json.
  2. Inside the Arduino IDE go to File > Preferences > Adicional board manager URLs.
  3. Add a line with this URL.
  4. Go to the IDE's board manager and install: Raspberry Pi Pico/RP2040/2350 by Early F. Philhower, III.



NEEEC-FEUP

Library Dependencies

- ***ArduinoJson by Benoit Blanchon***
  - Easy use of JSON/msgPack serialization.
- ***PubSubClient by Nick O'Leary***
  - Library that implements the MQTT features.

- These libraries can be installed on the Arduino IDE through the **Library Manager**

NEEEC-FEUP

The Beginning

```
#include <PubSubClient.h>
#include <WiFi.h>

#define SSID "wifi_name_123"
#define PASSWORD "wifi_password_123"
#define BROKER "broker.hivemq.com"

WiFiClient wifi;
PubSubClient mqtt(wifi);
```

NEEEC-FEUP

Initialization

```
void setup() {

  // connects to WiFi
  WiFi.begin(SSID, PASSWORD);
  while (WiFi.status() != WL_CONNECTED) delay(500);

  // prepares the PubSubClient
  mqtt.setServer(char *ip, uint16_t port);
  mqtt.setCallback(void (*callback)(char *topic, uint8_t *payload, unsigned int plength));
  mqtt.setBufferSize(uint16_t size)

}
```

NEEEC-FEUP

Ensuring Connections

```ino
void ensureWiFi() {
  if (WiFi.status() == WL_CONNECTED) return;

  while (WiFi.status() != WL_CONNECTED) {
    WiFi.begin(WIFI_SSID, WIFI_PASS);
    unsigned long start = millis(); // starts the timeout
    while (WiFi.status() != WL_CONNECTED && millis() - start < 10000) {
      delay(500);
      Serial.print(".");
    }
  }
  Serial.println(" CONNECTED TO WIFI!");
}
```

```ino
void ensureMQTT() {
  if (mqtt.connected()) return;

  while (!mqtt.connected()) {
    Serial.print(".");
    if (!mqtt.connect(MQTT_ID)) delay(1000);
  }
  Serial.println(" CONNECTED TO MQTT!");

  // only reached after a fresh connect, because of the early return above
  if (mqtt.subscribe(MQTT_SUB_TOPIC)) Serial.println(" SUBSCRIBED!");
  else Serial.println(" SUBSCRIBE FAILED");
}
```

NEEEC-FEUP

The Loop

```
void loop() {
  ensureWiFi();
  ensureMQTT();
  mqtt.loop();

  // we continue here
}
```

NEEEC-FEUP

Publishing and Subscribing

```
// publishes a message
bool mqtt.publish(const char *topic, const char *payload, bool retained = false);
bool mqtt.publish(const char *topic, const uint8_t *payload, unsigned int plength = false);

// subscribes to a topic
bool mqtt.subscribe(const char *topic);
```

NEEEC-FEUP

Callbacks & Lambdas

- A ***callback*** is a function that **is passed as a parameter** to another function.
    - **Purpose**: the receiving function calls the *callback* **later**, usually when an **event ocurrs**. It is what allows the code to *react*.
- A **Lambda Function** is a **nameless** function, defined in a **short** way and **in the place** where it is used.
    - **Purpose**: they’re frequently used to **create *callbacks*** quickly, without the need to declare a whole seperate function, making the code more concise.

NEEEC-FEUP

Callbacks and Receiving JSON/MsgPack

```
void callback(char *topic, uint8_t *payload, unsigned int plength) {
  JsonDocument doc;

  // deserializes Json or MsgPack into doc
  // pick only the one that applies
  DeserializationError err = deserializeJson(doc, payload, plength);
  // or
  DeserializationError err = deserializeMsgPack(doc, payload, plength);

  if (err) {
    Serial.print("Parse failed: ");
    Serial.println(err.c_str());
    return;
  }

  int number = doc["number"] | 0;
  float temp = doc["temperature"] | 0.0f;
  const char *name = doc["name"] | "N/A";

  Serial.printf("number = %d; temperature = %f; name = %s\n", number, temp, name);
}
```

NEEEC-FEUP

Publishing JSON/MsgPack

```
void publishMsgPack(const char *topic) {
  JsonDocument doc;
  doc["number"] = 147;
  doc["temperature"] = 21.6f;
  doc["name"] = "James";

  char buffer[128];
  size_t len = serializeJson(doc, buffer, sizeof(buffer));
  mqtt.publish(topic, buffer);

  uint8_t bin[128];
  size_t len = serializeMsgPack(doc, bin, sizeof(bin));
  mqtt.publish(topic, bin, len);
}
```

NEEEC-FEUP

Lambdas

```
mqtt.setCallback([](const char *topic, const uint8_t *payload, unsigned int plength){
  for (unsigned int i = 0; i < plength; i++) {
    Serial.print((char)payload[i]);
  }
});
```

NEEEC-FEUP

Lambdas

Inside the Capture Clause	Meaning
[]	**Does not** capture any external variables.
[var]	Captures a variable **by value** (a copy).
[&var]	Captures a variable **by reference** (direct access).
[=]	Captures **every** external variable in the scope **by value**.
[&]	Captures **every** external variable in the scope **by value**.

NEEEC-FEUP



Hands On Section

NEEEC-FEUP

1st Challenge

Broker: broker.hivemq.com
Topic: ws-mqtt/demo

EN - Send messages to the broker. For this challenge, publish a message confirming that your group has connected.

PT - Enviar mensagens para o broker. Neste desafio, enviem uma mensagem a dizer que o vosso grupo já se conectou.

canva.link/ 83fugywvrlz2i2q

SSID: iPhone
PASS: mqttworkshop

NEEEC-FEUP

2nd Challenge

Broker: broker.hivemq.com
Topic: ws-mqtt/chat

EN - Publish and subscribe to the topic and forward the output to the Pico's serial port with Serial.println().

PT - Publica e subscreve o tópico e reencaminha os dados para a porta serial da pico com Serial.println().

NEEEC-FEUP

3rd Challenge

Broker: broker.hivemq.com
Topic: ws-mqtt/requests

```
{
    “name”: “bob”
}
```

```
{
    “name”: “server”,
    “setLed”: true
}
```

EN - Send a message in the standard JSON format, with your device’s name, according to the structure. Then you will receive on “ws-mqtt/led/<name>” the second structure. Make the LED_BUILTIN react.

PT - Envia uma mensagem no formato JSON padrão, com o nome do teu dispositivo, de acordo com a estrutura. Em seguida, irás receber no tópico “ws-mqtt/led/<nome>” a segunda estrutura. Faz o LED_BUILTIN reagir.

NEEEC-FEUP

4º Challenge

Broker: broker.hivemq.com
Topic: ws-mqtt/sonar

```
{
    “dispositivo”: “bob”,
    “distancia_cm”: “47”,
    “local”: “FEUP/J202”
}
```

En - Use a Pico to publish sonar data in the standard JSON format and implement an LWT.

Pt - Usa uma Pico para publicar os dados no formato JSON padrão de um sonar e implementa um LWT.

NEEEC-FEUP

Thank you for coming to this workshop.

Credits:

Lourenço Marques, Vasco Mota, André Dias, João Carlos Lopes

NEEEC-FEUP

